



Trailhead



[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



[Get Started with Apex Triggers](#)

Get Started with Apex Triggers

Learning Objectives

After completing this unit, you'll be able to:

- Write a trigger for a Salesforce object.
- Use trigger context variables.
- Call a class method from a trigger.
- Use the sObject `addError()`  method in a trigger to restrict save operations.

Before You Begin

Apex triggers are useful, fun, and groovy. While this module helps you get started with them, it also references other Salesforce features to show you the power of Apex triggers. To get the most out of this module, we highly recommend you check out these modules first:

- [Quick Start: Apex](#)
(<https://trailhead.salesforce.com/content/learn/projects/quickstart-apex>)
- [Apex Basics and Database](#)
(https://trailhead.salesforce.com/content/learn/modules/apex_database)
- [SOQL for Admins](#) (<https://trailhead.salesforce.com/content/learn/modules/soql-for-admins>)
- [Developer Console Basics](#)
(https://trailhead.salesforce.com/content/learn/modules/developer_console)

Writing Apex Triggers

Apex triggers enable you to perform custom actions before or after events to records in Salesforce, such as insertions, updates, or deletions. Just like database systems support triggers, Apex provides trigger support for managing records.





Use triggers to perform tasks that can't be done by using the point-and-click tools in the Salesforce user interface. For example, if validating a field value or updating a field on a [Apex Trigger](https://trailhead.salesforce.com/content/learn/modules/apex_triggers/apex_triggers_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer), use a trigger. Use Apex triggers if performance and scale is important, if your logic is too complex for the point-and-click tools, or if you're executing

CPU-intensive operations.

Get Started with Apex Triggers

Triggers can be defined for top-level standard objects, such as Account or Contact, custom objects, and some standard child objects. Triggers are active by default when created. Salesforce automatically fires active triggers when the specified database events occur.

Trigger Syntax

The syntax of a trigger definition is different from a class definition's syntax. A trigger definition starts with the `trigger` keyword. It is then followed by the name of the trigger, the Salesforce object that the trigger is associated with, and the conditions under which it fires. A trigger has this syntax.

```
1 trigger TriggerName on ObjectName (trigger_events) {  
2     code_block  
3 }
```



To execute a trigger before or after insert, update, delete, and undelete operations, specify multiple trigger events in a comma-separated list. The events you can specify are:

- before insert
- before update
- before delete
- after insert
- after update
- after delete
- after undelete

Trigger Example

This simple trigger fires before you insert an account and writes a message to the debug log.

1. In the Developer Console, click **File | New | Apex Trigger**.



```
1 trigger HelloWorldTrigger on Account (before insert) {
2     System.debug('Hello World!');
3 }
```

[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

4. To save, press **Ctrl+S**.

Get Started with Apex Triggers

5. To test the trigger, create an account.

- Click **Debug | Open Execute Anonymous Window**.
- In the new window, add the following and then click **Execute**.

```
1 Account a = new Account(Name='Test Trigger');
2 insert a;
```

6. In the debug log, find the `Hello World!` statement. The log also shows that the trigger has been executed.

Types of Triggers

There are two types of triggers.

- **Before triggers** are used to update or validate record values before they're saved to the database.
- **After triggers** are used to access field values that are set by the system (such as a record's `Id` or `LastModifiedDate` field), and to affect changes in other records. The records that fire the *after trigger* are read-only.

Using Context Variables

To access the records that caused the trigger to fire, use context variables. For example, `Trigger.new` contains all the records that were inserted in insert or update triggers. `Trigger.old` provides the old version of sObjects before they were updated in update triggers, or a list of deleted sObjects in delete triggers. Triggers can fire when one record is inserted, or when many records are inserted in bulk via the API or Apex. Therefore, context variables, such as `Trigger.new`, can contain only one record or multiple records. You can iterate over `Trigger.new` to get each individual sObject.

This example is a modified version of the `HelloWorldTrigger` example trigger. It iterates over each account in a for loop and updates the `Description` field for each.



5 | }

Apex Triggers (/content/learn/modules/apex_triggers?

trailmix_creator_id=srebello7&trailmix_slug=salesforce... The system saves the records that fired the before trigger after the trigger finishes execution. You can modify the records in the trigger without explicitly calling a DML insert or update operation. If you perform DML statements on those records, you get an error.



Get Started with Apex Triggers

Some other context variables return a Boolean value to indicate whether the trigger was fired due to an update or some other event. These variables are useful when a trigger combines multiple events. For example:

```
1 trigger ContextExampleTrigger on Account (before insert, after
2 insert, after delete) {
3     if (Trigger.isInsert) {
4         if (Trigger.isBefore) {
5             // Process before insert
6         } else if (Trigger.isAfter) {
7             // Process after insert
8         }
9     }
10    else if (Trigger.isDelete) {
11        // Process after delete
12    }
13 }
```



This table offers a comprehensive list of all context variables available for triggers.

Variable	Usage
<code>isExecuting</code> copy	Returns true if the current context for the Apex code is a trigger, not a Visualforce page, a web service, or an <code>executeanonymous()</code> API call.
<code>isInsert</code> copy	Returns <code>true</code> if this trigger was fired due to an insert operation, from the Salesforce user interface, Apex, or the API.


[copy](#)

from the Salesforce user interface, Apex, or the API.

[Apex Triggers \(/content/learn/modules/apex_triggers?](/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce)

[trailmix_creator_id=srebello7&trailmix_slug=salesforce">is created](#) to a delete operation,

[copy](#)

from the Salesforce user interface, Apex, or the API.

Get Started with Apex Triggers

[isBefore](#)
[copy](#)

Returns [true](#) [copy](#) if this trigger was fired before any record was saved.

[isAfter](#)
[copy](#)

Returns [true](#) [copy](#) if this trigger was fired after all records were saved.

[isUndelete](#)
[copy](#)

Returns [true](#) [copy](#) if this trigger was fired after a record is recovered from the Recycle Bin. This recovery can occur after an undelete operation from the Salesforce user interface, Apex, or the API.

[new](#)
[copy](#)

Returns a list of the new versions of the sObject records.

This sObject list is only available in [insert](#) [copy](#), [update](#) [copy](#), and [undelete](#) [copy](#) triggers, and the records can only be modified in [before](#) [copy](#) triggers.

[newMap](#)
[copy](#)

A map of IDs to the new versions of the sObject records.

This map is only available in [before update](#) [copy](#), [after insert](#) [copy](#), [after update](#) [copy](#), and [after undelete](#) [copy](#) triggers.

[old](#)
[copy](#)

Returns a list of the old versions of the sObject records.

This sObject list is only available in [update](#) [copy](#) and [delete](#) [copy](#) triggers.

[oldMap](#)
[copy](#)

A map of IDs to the old versions of the sObject records.

This map is only available in [update](#) [copy](#) and [delete](#) [copy](#) triggers.



copy	current operation.
Apex Triggers (/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)	Possible values of the <code>System.TriggerOperation</code> enum are: <code>BEFORE_INSERT</code>, <code>BEFORE_UPDATE</code>, <code>BEFORE_DELETE</code>, <code>AFTER_INSERT</code>, <code>AFTER_UPDATE</code>, <code>AFTER_DELETE</code>, and <code>AFTER_UNDELETE</code>. If you vary your programming logic based on different trigger types, consider using the <code>switch</code> statement with different permutations of unique trigger execution enum states.
size	The number of records processed in a trigger invocation. DML operations that include over 200 records are processed in batches, and the trigger is invoked for each batch. <code>Trigger.size</code> includes only the number of records in the current batch, not the total number of records in the DML operation.

Calling a Class Method from a Trigger

You can call public utility methods from a trigger. Calling methods of other classes enables code reuse, reduces the size of your triggers, and improves maintenance of your Apex code. It also allows you to use object-oriented programming.

The following example trigger shows how to call a static method from a trigger. If the trigger was fired because of an insert event, the example calls the static `notifyUsers()` method on the `CustomContactNotification` class. This utility method sends a custom notification to the Salesforce users defined in the trigger. The notification contains the number of contact records inserted.



For this example, you create a custom notification. Custom notifications are available only in Lightning Experience. Creating and editing notifications requires the Customize Application user permission. For more information, see [Create a Desktop or Mobile Notification \(https://help.salesforce.com/s/articleView?id=platform.notif_builder_custom_type.htm&language=en_US&type=5\)](https://help.salesforce.com/s/articleView?id=platform.notif_builder_custom_type.htm&language=en_US&type=5).

1. Create a desktop custom notification.

- From Setup, in the Quick Find box, enter `Notification Builder`, and then select **Custom Notifications**.
- Click **New**.



2. In the Developer Console, click **File | New | Apex Class**.

3. Enter `CustomContactNotification` , and then click **OK**.

Apex Triggers - content from /modules/apex-triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) `CustomContactNotification`  class

Get Started with Apex Triggers

```

1  public with sharing class CustomContactNotification {
2      // Public method
3      public static void notifyUsers(Set<String> recipientsIds,
4  Integer recordCount) {
5          // Get the ID for the custom notification type created
6  in Setup
7          CustomNotificationType notificationType =
8              [SELECT Id, DeveloperName
9              FROM CustomNotificationType
10             WHERE DeveloperName='New_Contact_Notification'];
11         // Create a new custom notification
12         Messaging.CustomNotification notification = new
13         Messaging.CustomNotification();
14         // Set the contents for the notification
15         notification.setTitle('Trailhead Trigger Tutorial');
16         notification.setBody(recordCount + ' contact(s) were
17         inserted.');
```



Show More

5. In the Developer Console, click **File | New | Apex Trigger**.

6. Enter `ContactNotificationTrigger`  for the trigger name, and then select **Contact** for the sObject. Click **Submit**.

7. Replace the default code with the following.

```

1  trigger ContactNotificationTrigger on Contact (after insert,
2  after delete) {
3      if (Trigger.isInsert) {
4          Integer recordCount = Trigger.new.size();
5          // Set the recipientIDs to the current user
```





```

11     }
12     else if (Trigger.isDelete) {

```

[Apex Triggers \(/content/learn/modules/apex-triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

```

}

```

Get Started with Apex Triggers

8. To save, press **Ctrl+S**.

9. To test the trigger, create a contact.

- Click **Debug | Open Execute Anonymous Window**.
- In the new window, add the following and then click **Execute**.

```

1 Contact c = new Contact (LastName='Test Contact');
2 insert c;

```



10. In the debug log, check that the trigger was fired. Toward the end of the log, find the debug message that was written by the utility method: `DEBUG|Custom notification sent successfully.` [copy](#)
11. Now check that you received the notification. To open the Notification panel, click . You should see a notification with the title “Trailhead Trigger Tutorial” and the body “1 contact(s) were inserted.”

With your new trigger in place, you get a custom desktop notification every time you add one or more contacts!

Adding Related Records

Triggers are often used to access and manage records related to the records in the trigger context—the records that caused this trigger to fire.

This trigger adds a related opportunity for each new or updated account if no opportunity is already associated with the account. The trigger first performs a SOQL query to get all child opportunities for the accounts that the trigger fired on. Next, the trigger iterates over the list of sObjects in `Trigger.new` [copy](#) to get each account sObject. If the account doesn’t have any related opportunity sObjects, the for loop creates one. If the trigger created any new opportunities, the final statement inserts them.



```

1 trigger AddRelatedRecord on Account (after insert, after update) {
2
3     List<Opportunity> oppList = new List<Opportunity>();
4     // Get the related opportunities for the accounts in this
5     trigger
6     Map<Id, Account> acctsWithOpps = new Map<Id, Account>(
7         [SELECT Id, (SELECT Id FROM Opportunities) FROM Account
8         WHERE Id IN :Trigger.new]);
9         // Add an opportunity for each account if it doesn't already
10        have one.
11        // Iterate through each account.
12        for (Account a : Trigger.new) {
13
14            System.debug('acctsWithOpps.get(a.Id).Opportunities.size()=' +
15            acctsWithOpps.get(a.Id).Opportunities.size());
16            // Check if the account already has a related
17            opportunity.
18            if (acctsWithOpps.get(a.Id).Opportunities.size() == 0) {
19                // If it doesn't, add a default opportunity
20                oppList.add(new Opportunity(Name=a.Name + '

```

[Apex Triggers \(/content/learn/modules/apex_triggers/apex_triggers_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/apex_triggers/apex_triggers_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Get Started with Apex Triggers

Show More

2. To test the trigger, create an account in the Salesforce user interface and name it

Apples and Oranges [copy](#).

3. In the Opportunities related list on the account's page, find the new opportunity. The trigger added this opportunity automatically!



The trigger you've added iterates over all records that are part of the trigger context—the for loop iterates over `Trigger.new` [copy](#). However, the loop in this trigger could be more efficient. We don't really need to access every account in this trigger context, but only a subset—the accounts without opportunities. The next unit shows how to make this trigger more efficient. In the Bulk Trigger Design Patterns unit, learn how to modify the SOQL query to get only the accounts with no opportunities. Then, learn to iterate only over those records.



`addError()`  method throws a fatal error inside a trigger. The error message is displayed in the user interface and is logged.

[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

The following trigger prevents the deletion of an account if it has related opportunities.

By default, deleting an account causes a cascade delete of all its related records. This ~~Get started with Apex Triggers~~ cascade delete of opportunities. Try this trigger for yourself! If

you've executed the previous example, your org has an account called `Apples and Oranges`  with a related opportunity. This example uses that sample account.

1. Using the Developer Console, add the following trigger.

```
1 trigger AccountDeletion on Account (before delete) {
2     // Prevent the deletion of accounts if they have related
3     opportunities.
4     for (Account a : [SELECT Id FROM Account
5                       WHERE Id IN (SELECT AccountId FROM
6 Opportunity) AND
7                               Id IN :Trigger.old]) {
8         Trigger.oldMap.get(a.Id).addError(
9             'Cannot delete account with related
opportunities.');
```



2. In the Salesforce user interface, navigate to the `Apples and Oranges`  account's page and click **Delete**.

3. In the confirmation popup, click **Delete**.

Find the validation error with the custom error message `Cannot delete account with related opportunities` .

4. Disable the `AccountDeletion`  trigger. If you leave this trigger active, you can't check your challenges.

- From Setup, search for `Apex Triggers` .
- On the Apex Triggers page, click **Edit** next to the `AccountDeletion`  trigger.
- Deselect **Is Active**.
- Click **Save**.



the runtime engine sets the bad records aside. The runtime engine then attempts a partial save of the records that did not generate

errors.

[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

If a DML statement in Apex spawned the trigger, any error rolls back the entire operation. However, the runtime engine still processes

records that were not in the operation to compile a comprehensive list of

Get Started with Apex Triggers

errors.

Triggers and Callouts

Apex allows you to make calls to and integrate your Apex code with external web services. Apex calls to external web services are referred to as callouts. For example, you can make a callout to a stock quote service to get the latest quotes. When making a callout from a trigger, the callout must be done asynchronously so that the trigger process doesn't block you from working while waiting for the external service's response. The asynchronous callout is made in a background process, and the response is received when the external service returns it.

To make a callout from a trigger, call a class method that executes asynchronously. Such a method is called a future method and is annotated with `@future(callout=true)` [copy](#). This example class contains the future method that makes the callout.



The example uses a hypothetical endpoint URL for illustration purposes only. You can't run this example unless you change the endpoint to a valid URL and add a remote site in Salesforce for your endpoint.

```
1 public class CalloutClass {
2     @future(callout=true)
3     public static void makeCallout() {
4         HttpRequest request = new HttpRequest();
5         // Set the endpoint URL.
6         String endpoint = 'http://yourHost/yourService';
7         request.setEndPoint(endpoint);
8         // Set the HTTP verb to GET.
9         request.setMethod('GET');
10        // Send the HTTP request and get the response.
11        HttpResponse response = new HTTP().send(request);
12    }
```





asynchronously.

[Apex Triggers \(https://content.trailhead.salesforce.com/learn/modules/apex_triggers/apex_triggers_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://content.trailhead.salesforce.com/learn/modules/apex_triggers/apex_triggers_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) (before insert, before update)



3 `CalloutClass.makeCallout();`

Get Started with Apex Triggers

This section offers only an overview of callouts and is not intended to cover callouts in detail. For more information, see [Invoking Callouts Using Apex](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts.htm) in the Apex Developer Guide.

Resources

- [Apex Developer Guide: Triggers](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers.htm).
- [Apex Developer Guide: Invoking Callouts Using Apex](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts.htm).
- [Apex Reference Guide: CustomNotification Class](https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_class_Messaging_CustomNotification.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_class_Messaging_CustomNotification.htm).
- [Trailhead: Apex Integration Services](https://trailhead.salesforce.com/content/learn/modules/apex_integration_services) (https://trailhead.salesforce.com/content/learn/modules/apex_integration_services).

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Create an Apex trigger



Pre-Work:

Apex Triggers (/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

And a checkbox field to the Account object:

- Field Label: Match Billing Address copy

Get Started with Apex Triggers

Match_Billing_Address copy

Note: The resulting API Name should be Match_Billing_Address__c

copy .

- Create an Apex trigger:
 - Name: AccountAddressTrigger copy
 - Object: **Account**
 - Events: before insert and before update
 - Condition: Match Billing Address is true
 - Operation: set the Shipping Postal Code to match the Billing Postal Code

Choose hands-on org

Koushik nelluri Salesforce CRM

Created on 5/9/2026



Launch

Check Challenge to Earn 500 Points

Learn

Badges

Trails

Trailmixes

Superbadges

Trailhead Academy

Certifications

Certifications

Maintain Certifications

Verify Certifications

Community

Trailblazer Community

Events

Agentblazers

Salesforce Developers

Salesforce Admins

Extras

Trailhead Login

Sales Enablement

Trailblazer Store

Salesforce Help



Download Trailhead GO



Want to learn more about Apex Triggers?
Trailmix Creator ID: srebello7 (Trailmix slug: salesforce-platform-developer).

Get Started with Apex Triggers



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)



Engli



Your Privacy Choices